Java Algorithms Project
Project Requirements

Minimum requirements - cipher algorithms

If you build a cipher algorithms project, any items marked incomplete in this section will result in the project receiving a failing grade.

	Complete?
Ceasar cipher	
Users can input a message and a key to encrypt a message by shifting each letter in the message by the key.	
Users can input a message and a key to decrypt a message by shifting each letter in the message by the key.	
Users can undo the last encryption using a stack-based undo system.	
Users can undo the last decryption using a stack-based undo system.	
Users can input non-alphabetic characters in the message, and they are not encrypted.	
Users can input a message with both uppper and lower case letters, and the case is preserved in the encrypted message.	
Users can see an error message if they input an invalid key or message.	
Vigenere cipher	
Users can input a message and a key to encrypt a message by shifting each letter in the message by the corresponding letter in the key.	
Users can input a message and a key to decrypt a message by shifting each letter in the message by the corresponding letter in the key.	
Keywords are repeated if the message is longer than the key recursively.	
Users can see an error message if they input an invalid key or message.	
Users can perform a frequency analysis on the message to determine the most likely key.	
Users can input non-alphabetic characters in the message, which are not encrypted.	
Users can input a message with both upper and lowercase letters, and the case is preserved in the encrypted message.	
Users can see an error message if they input an invalid key or message.	
Overall requirements	
Users can start the app by running the ***main*** method in the ***App*** class.	
Users can use the ***help*** subcommand to see a list of commands.	
The app is deployed online so that the rest of the world can use it.	
You are shown as the only contributor to the project on GitHub.	

Minimum requirements - maze generator and solver

If you build a maze generator and solver project, any items marked incomplete in this section will result in the project receiving a failing grade.

Maze gengeration and solver	
Users can input the height and width to generate a maze.	
Users can input mathematical expressions to customize the maze generation.	
Users can input ***dfs*** to solve the maze using a depth-first search algorithm.	
Users can input ***bfs*** to solve the maze using a breadth-first search algorithm.	
Users can input ***a**** to solve the maze using an A* or Dijkstra's search algorithm.	
Users can use ***all*** to solve the maze using all three algorithms and see the output of the common path between the three algorithms or a message of `No common path`.	
Users can manipulate the maze by adding or removing walls.	
Users can save the maze as a binary string to a file.	
Users can load a maze from a file.	
Users can see an error message if they input an invalid command.	
Overall requirements	
Users can start the app by running the ***main*** method in the ***App*** class.	
Users can use the ***help*** subcommand to see a list of commands.	
The app is deployed online so that the rest of the world can use it.	
You are shown as the only contributor to the project on GitHub.	

Convention requirements

More than two items marked incomplete in this section will result in the project receiving a failing grade.

	Complete?
The files in the app are organized following the conventions demonstrated in lectures.	
The code in the app adheres to coding conventions covered in lessons, like using plural names for arrays.	
The app does not contain dead code, commented-out sections, or console logs.	
The app can be used without encountering errors.	
The app is coded using proper indentation.	
The GitHub repository used for the project must be named appropriately. For example, names that describe your project are appropriate, whereas generic names such as ***ga-project*** or ***unit-project*** are not.	

README requirements

More than two items marked incomplete in this section will result in the project receiving a failing grade.

	Complete?
The app's name.	
A brief description of your app and its functionality.	
A Getting started section. This section should include instructions on how to use your app.	
A Technologies used section. This section should list the principal technologies used by your application, including any major frameworks or libraries.	
If necessary, an Attributions section. This section should include links to any external resources (such as libraries or assets) you used to develop your application that require attribution. You can exclude this section if it does not apply to your application.	

Summary

The project meets the requirements above and passes.	